

M2 명령줄 인터페이스 설계

2026. 10. 09.

목차

1. 요약
2. 배경과 목표
3. 설계
4. 구현 계획
5. 평가 방법
6. 위험과 대응

참고 출처

문서의 목적. M1의 `convert()`를 셸에서 `python convert.py 100 km mi`로 부를 수 있게 하는 명령줄 인터페이스의 입력 해석, 출력 형식, 오류 처리, 시험 방법을 설명하고, 구현 전에 방식과 완료 기준의 승인을 받는다.

1 요약

M2는 `convert.py`에 `main()` 하나를 더해 명령줄에서 쓸 수 있게 한다. 변환 계산은 M1 그대로다[D1].

- **입력.** `argparse`로 값·원래 단위·바꿀 단위를 받는다. 값은 유한한 숫자만 받는다. 미리 시험해 보니 `nan`과 `inf`가 그대로 통과했기 때문이다[D3].
- **출력.** 유효숫자 10자리와 단위를 출력한다(62.13711922 mi). 이렇게 하면 211.99999999999994 같은 부동소수점 잔여가 212로 보인다.
- **오류.** 모든 오류는 `argparse`의 방식으로 통일한다. 표준 오류 출력에 사용법 한 줄과 오류 한 줄을 쓰고 종료 코드 2로 끝낸다.
- **시험.** 실제 프로세스를 띄워 종료 코드와 출력을 확인하는 시험 10건을 추가한다.

2 배경과 목표

M1에서 `convert(value, src, dst)`가 완성되었다. 시험 127건이 통과했고 `import`는 0개다[D1]. 지금은 Python 코드 안에서만 부를 수 있다. M2의 완료 기준은 프로포절 표 5에 있다[D2]. 표 1은

M1에서 넘겨받은 것과 M2가 할 일이다.

표 1. M1에서 넘겨받은 것과 M2가 할 일. [D1, D2]

항목	M1 상태	M2에서
변환	convert(), 정확도 시험 완료	그대로 호출
오류 3종	ValueError와 메시지	잡아서 종료 코드 2
숫자 아닌 값	M2로 넘김	입력 해석에서 거부
단위 표기	오류 메시지는 소문자(개발자 결정, 2026.10.09)	그대로 유지

3 설계

그림 1은 M1이 끝난 지금(a)과 M2 후(b)의 호출 흐름이다. 새로 생기는 것은 입력 해석과 출력 두 부분이다.

(a) 지금 (M1 후)



(b) M2 후

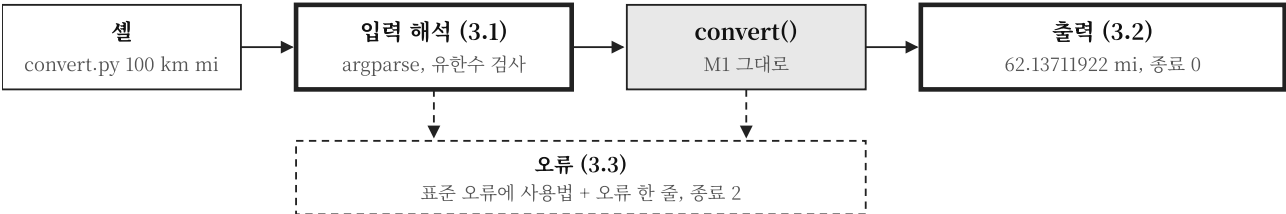


그림 1. 지금(a)과 M2 후(b)의 흐름. 굵은 테두리가 새로 만드는 부분, 회색이 M1에서 그대로 쓰는 부분, 점선이 오류 경로다. 오류는 입력 해석과 변환 두 곳에서 생기지만 같은 형식으로 끝난다.

3.1 입력 해석

위치 인자 세 개 value src dst를 받는다. value는 문자열을 숫자로 바꾸는 함수를 거친다. 이 함수는 float()로 바꾼 뒤 math.isfinite()로 유효수인지 확인하고, 아니면 argparse.ArgumentTypeError를 낸다[R1, R2]. float()은 "nan", "inf"를 숫자로 받아들이므로 유효수 검사가 따로 필요하다. 표 2는 Python 3.11.16에서 argparse가 실제로 어떻게 해석하는지 미리 확인한 결과다[D3].

표 2. 입력별 argparse 해석 (기본 float 변환, 사전 확인). [D3, R1]

입력	argparse 결과	M2에서
-40 c f	value = -40.0	그대로 (음수 정상)
-1.5 c f	value = -1.5	그대로
abc km mi	종료 2, "invalid float value: 'abc'"	그대로 (숫자 아닌 값 거부)
nan km mi	value = nan (통과)	유한수 검사로 거부
inf km mi	value = inf (통과)	유한수 검사로 거부
-1e3 m km	종료 2, "invalid float value: 'm'" (오해하기 쉬운 메시지)	도움말에 -- 사용 안내 (3.3절)
-- -1e3 m km	value = -1000.0	그대로

3.2 출력

결과는 `f"{result:.10g} {dst}"` 형식으로 표준 출력에 한 줄 쓴다. `.10g`는 유효숫자 10자리로 반올림하고, 필요 없는 0과 소수점은 지우며, 아주 크거나 작은 값은 지수 표기로 바꾼다[R3]. 단위는 사용자가 입력한 그대로 보인다(`100 c F` → `212 F`). 표 3은 실제 계산값이 어떻게 보이는지다.

표 3. 출력 형식 예. Python 3.11.16에서 확인. [D3]

계산값	출력	설명
62.13711922373339	62.13711922	유효숫자 10자리
211.99999999999994	212	부동소수점 잔여가 사라짐
-40.000000000000006	-40	같음
1000000.0	1000000	필요 없는 ".0" 제거
1.5e+20 / 1e-07	1.5e+20 / 1e-07	큰 값·작은 값의 지수 표기

3.3 오류와 도움말

`convert()`가 낸 `ValueError`는 `parser.error(메시지)`로 넘긴다. `argparse`가 자기 오류(숫자 아닌 값 등)를 낼 때와 같은 방식이다. 표준 오류에 사용법 한 줄과 `convert.py: error: ...` 한 줄을 쓰고 종료 코드 2로 끝난다[R1]. 프로포절은 “한 줄 메시지”라고 적었는데[D2], **오류 형식을 하나로 맞추기 위해 argparse 관례대로 사용법 한 줄이 함께 붙는다**. 이렇게 하면 오류는 모두 같은 모양이 되고 종료 코드도 하나가 된다.

`-h` 도움말 끝에는 쓸 수 있는 단위 목록(M1의 표에서 만듦)과, 지수 표기 음수는 `--`를 앞에 붙이라는 안내를 넣는다. Python 3.14부터는 `-1e3`도 음수로 인식되므로 이 안내가 필요 없어진다[R1].

4 구현 계획

새 파일은 없다. 새 의존성은 표준 라이브러리 `argparse`와 `math` 두 개다. 표 4가 변경이다.

표 4. 바뀌는 파일.

파일	변경	이유
<code>convert.py</code>	유한수 변환 함수, <code>main(argv=None)</code> , <code>if __name__ == "__main__"</code> 블록. <code>import argparse, math</code>	3.1-3.3절
<code>test_convert.py</code>	CLI 시험 묶음 추가. <code>subprocess</code> 로 실제 실행	종료 코드와 출력 확인

- ① 시험 먼저. 표 5의 CLI 시험을 적고 실패를 확인한다.
- ② `main()` 작성. 시험을 통과시킨다.
- ③ 전체 실행. M1 시험 7개와 함께 모두 돌리고, 셸에서 직접 몇 가지를 실행해 출력을 눈으로 확인한다.

5 평가 방법

CLI 시험은 `subprocess.run([sys.executable, "convert.py", ...])`로 실제 프로세스를 띄워 표준 출력, 표준 오류, 종료 코드를 확인한다[R4]. 함수만 부르는 시험으로는 종료 코드와 셸 인자 해석을 확인할 수 없기 때문이다. 표 5는 시험 경우와 완료 기준이다.

표 5. CLI 시험과 완료 기준. 프로포절 표 4·5의 M2 기준을 포함한다. [D2]

묶음	입력 → 기대	목표
정상 출력	<code>100 km mi → 62.13711922 mi</code> , <code>-40 C F → -40 F</code> , <code>-- -1e3 m km → -1 km</code> ; 모두 종료 0	3 / 3
오류	없는 단위(1 km xx), 종류 불일치(1 km kg), 숫자 아님(abc km mi), 0 K 미만 (<code>-300 c f</code>), <code>nan</code> , <code>inf</code> ; 모두 종료 2, 표준 출력 비어 있음, 표준 오류에 이유	6 / 6
도움말	<code>-h</code> → 종료 0, 단위 목록과 <code>--</code> 안내 포함	1 / 1
회귀	M1 시험 7개	7 / 7
의존성	<code>convert.py</code> import	<code>argparse</code> , <code>math</code> 만

6 위험과 대응

표 6은 M2에서 생길 수 있는 문제와 그것을 알아채는 방법이다.

표 6. 위험과 알아채는 방법.

위험	알아채는 방법	대응
-1e3의 오해하기 쉬운 오류	표 2에서 이미 확인함(“invalid float value: 'm'”)	도움말 안내로 대응한다. 인자를 직접 해석하는 우회는 코드가 늘어 하지 않는다. 3.14 이상에서는 저절로 해결된다[R1].
시험 실행 위치에 따라 convert.py를 못 찾음	다른 폴더에서 시험을 돌리면 CLI 시험이 실패한다.	시험 파일 위치를 기준으로 경로를 만든다.
Windows 콘솔 인코딩	출력이 숫자와 ASCII 단위 기호뿐이라 영향이 없을 것으로 본다(검증 안 함).	°같은 기호를 출력하지 않는다.

참고 출처

- [R1] Python Software Foundation. argparse — Parser for command-line options, arguments and subcommands (“Arguments containing -”, ArgumentParser.error, type).
<https://docs.python.org/3/library/argparse.html>
- [R2] Python Software Foundation. math.isfinite.
<https://docs.python.org/3/library/math.html#math.isfinite>
- [R3] Python Software Foundation. Format Specification Mini-Language (“g” presentation type).
<https://docs.python.org/3/library/string.html#format-specification-mini-language>
- [R4] Python Software Foundation. subprocess.run.
<https://docs.python.org/3/library/subprocess.html#subprocess.run>
- [D1] M1 변환 핵심 결과 (docs/reports/unit-converter-cli/M1-result.pdf), 2026.10.09 승인.
- [D2] 단위 변환 CLI 프로포절 (docs/reports/unit-converter-cli/00-proposal.pdf), 2026.10.09 승인.
- [D3] argparse 해석과 출력 형식 사전 확인, Python 3.11.16, 2026.10.09.